PRACTICAL 1(a): HELLO WORLD ANDROID APPLICATION

Project Name: HelloWorld
Template: Empty Views Activity
Language: Javagive me the

1. MainActivity.java

```
package com.example.helloworld;


import android.os.Bundle;

import androidx.appcompat.app.AppCompatActivity;


public class MainActivity extends AppCompatActivity {


    @Override

    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_main);

    }

}
```

2. activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout

    xmlns:android="http://schemas.android.com/apk/res/android"

    android:layout_width="match_parent"

    android:layout_height="match_parent"

    android:orientation="vertical"

    android:gravity="center"

    android:padding="16dp">


    <TextView
```

```
    android:id="@+id/text_hello"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="6137"
    android:textSize="24sp"
    android:textStyle="bold"
    android:textColor="#000000" />

</LinearLayout>
```

3. AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

  <application
    android:allowBackup="true"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:supportsRtl="true"
    android:theme="@style/Theme.Helloworld">

    <activity
      android:name=".MainActivity"
      android:exported="true">

      <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
```

```
            </intent-filter>

        </activity>

    </application>

</manifest>
```

4. colors.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFFF</color>
    <color name="purple_500">#6200EE</color>
    <color name="purple_700">#3700B3</color>
</resources>
```

5. styles.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>

    <style name="Theme.Helloworld"
        parent="Theme.MaterialComponents.DayNight.DarkActionBar">

        <item name="colorPrimary">@color/purple_500</item>
        <item name="colorPrimaryVariant">@color/purple_700</item>
        <item name="colorOnPrimary">@color/white</item>

    </style>
```

</resources>

Expected Output: The application displays 6137 in bold text at the centre of the screen.

PRACTICAL 1(b): PROGRAMMING RESOURCES IN ANDROID

Project Name: Practical2R
Template: Empty Views Activity
Language: Java

1. AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/AppTheme"
        tools:targetApi="31">

        <activity
            android:name=".ThemeActivity"
            android:label="Theme Demo" />

        <activity
            android:name=".StringActivity"
            android:label="String Demo" />
```

```
        <activity
            android:name=".DrawableActivity"
            android:label="Drawable Demo" />

        <activity
            android:name=".DimensionActivity"
            android:label="Dimension Demo" />

        <activity
            android:name=".ColorActivity"
            android:label="Color Demo" />

        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:label="@string/app_name">

            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>

        </activity>

    </application>

</manifest>
```

2. MainActivity.java

```
package com.example.practical2r;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }

    public void openThemeActivity(View view) {
        startActivity(new Intent(this, ThemeActivity.class));
    }

    public void openStringActivity(View view) {
        startActivity(new Intent(this, StringActivity.class));
    }

    public void openDrawableActivity(View view) {
        startActivity(new Intent(this, DrawableActivity.class));
    }

    public void openDimensionActivity(View view) {
```

```
        startActivity(new Intent(this, DimensionActivity.class));
    }

    public void openColorActivity(View view) {
        startActivity(new Intent(this, ColorActivity.class));
    }
}
```

3. ColorActivity.java

```
package com.example.practical2r;

import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;

public class ColorActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_color);
    }
}
```

4. ThemeActivity.java

```
package com.example.practical2r;

import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;

public class ThemeActivity extends AppCompatActivity {
```

```
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_theme);
    }
}
```

5. StringActivity.java

```
package com.example.practical2r;

import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;

public class StringActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_string);
    }
}
```

6. DrawableActivity.java

```
package com.example.practical2r;

import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;

public class DrawableActivity extends AppCompatActivity {
```

```
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_drawable);
    }
}
```

7. DimensionActivity.java

```
package com.example.practical2r;

import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;

public class DimensionActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_dimension);
    }
}
```

8. activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
```

```
android:padding="16dp"
tools:context=".MainActivity">

<Button
    android:id="@+id/btnColor"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Color Demo"
    android:layout_centerHorizontal="true"
    android:layout_marginTop="30dp"
    android:onClick="openColorActivity" />

<Button
    android:id="@+id/btnTheme"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Theme Demo"
    android:layout_below="@id/btnColor"
    android:layout_centerHorizontal="true"
    android:layout_marginTop="20dp"
    android:onClick="openThemeActivity" />

<Button
    android:id="@+id/btnString"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="String Demo"
    android:layout_below="@id/btnTheme"
```

```
    android:layout_centerHorizontal="true"
    android:layout_marginTop="20dp"
    android:onClick="openStringActivity" />

  <Button
    android:id="@+id/btnDrawable"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Drawable Demo"
    android:layout_below="@id/btnString"
    android:layout_centerHorizontal="true"
    android:layout_marginTop="20dp"
    android:onClick="openDrawableActivity" />

  <Button
    android:id="@+id/btnDimension"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Dimension Demo"
    android:layout_below="@id/btnDrawable"
    android:layout_centerHorizontal="true"
    android:layout_marginTop="20dp"
    android:onClick="openDimensionActivity" />

</RelativeLayout>
```

9. activity_color.xml

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
```

```
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/colorRed">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Color Activity"
        android:textSize="24sp"
        android:layout_centerInParent="true"
        android:textColor="@color/white" />

</RelativeLayout>
```

10. activity_dimension.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp"
    android:gravity="center">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Dimension Activity"
```

```
    android:textSize="24sp"
    android:textColor="@android:color/black" />

  <Button
    android:id="@+id/dimensionButton"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Click Me!"
    android:textSize="@dimen/button_text_size"
    android:padding="@dimen/button_padding"
    android:layout_marginTop="@dimen/button_margin_top"
    android:background="@color/btn_color" />

</LinearLayout>
```

11. activity_theme.xml

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
  xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:background="@android:color/holo_blue_bright">

  <TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="You got this sky blue theme"
    android:textSize="24sp"
    android:layout_centerInParent="true"
```

```
        android:textColor="@android:color/black" />


</RelativeLayout>
```

12. activity_string.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:background="@color/colorPrimaryDark">


    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/Welcome_Message"
        android:textSize="24sp"
        android:textColor="@android:color/white" />


</LinearLayout>
```

13. activity_drawable.xml

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@drawable/pebbles">
```

```
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Drawable Activity"
        android:textSize="24sp"
        android:layout_centerInParent="true"
        android:textColor="@android:color/white" />

</RelativeLayout>
```

14. colors.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="purple_200">#FFBB86FC</color>
    <color name="purple_500">#FF6200EE</color>
    <color name="purple_700">#FF3700B3</color>
    <color name="teal_200">#9ADA03</color>
    <color name="teal_700">#FF018786</color>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFFF</color>
    <color name="btn_color">#FFC49662</color>
    <color name="colorPrimary">#6200EE</color>
    <color name="colorPrimaryDark">#3700B3</color>
    <color name="colorAccent">#03DAC5</color>
    <color name="colorRed">#FFFF0000</color>
</resources>
```

15. dimens.xml

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<resources>
    <dimen name="button_margin_top">24dp</dimen>
    <dimen name="button_padding">16dp</dimen>
    <dimen name="button_text_size">18sp</dimen>
</resources>
```

16. strings.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="app_name">Practical2R</string>
    <string name="ParaHead">Welcome to Practical 2R</string>
    <string name="Welcome_Message">6137</string>
</resources>
```

17. themes.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>

    <style name="AppTheme"
        parent="Theme.AppCompat.Light.DarkActionBar">

        <item name="colorPrimary">@color/colorPrimary</item>
        <item name="colorPrimaryDark">@color/colorPrimaryDark</item>
        <item name="colorAccent">@color/colorAccent</item>

    </style>

</resources>
```

Drawable resource: Add an image named pebbles.png or pebbles.jpg to app/src/main/res/drawable/. The layouts use @drawable/pebbles, so the image is required.

Expected Output: The app displays buttons for Color, Theme, String, Drawable and Dimension demos. Clicking a button opens its corresponding screen.

PRACTICAL 1(c): ACTIVITY LIFECYCLE USING LOGCAT

Project Name: LogCatDemo
Template: Empty Views Activity
Language: Java

1. MainActivity.java

```
package com.example.logact;


import android.os.Bundle;

import android.util.Log;

import androidx.appcompat.app.AppCompatActivity;


public class MainActivity extends AppCompatActivity {


    @Override

    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_main);

        Log.d("ActivityLifecycle", "onCreate invoked");

    }


    @Override

    protected void onStart() {

        super.onStart();

        Log.d("ActivityLifecycle", "onStart invoked");

    }
```

```
    @Override
    protected void onResume() {
        super.onResume();
        Log.d("ActivityLifecycle", "onResume invoked");
    }

    @Override
    protected void onPause() {
        super.onPause();
        Log.d("ActivityLifecycle", "onPause invoked");
    }

    @Override
    protected void onStop() {
        super.onStop();
        Log.d("ActivityLifecycle", "onStop invoked");
    }

    @Override
    protected void onDestroy() {
        super.onDestroy();
        Log.d("ActivityLifecycle", "onDestroy invoked");
    }
}
```

2. activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Activity Lifecycle Demo"
        android:textSize="20sp" />

</LinearLayout>
```

3. AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <application
        android:allowBackup="true"
        android:label="LogCatDemo"
        android:supportsRtl="true"
        android:theme="@style/Theme.LogCatDemo">

        <activity
            android:name=".MainActivity"
            android:exported="true">

            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
```

```
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>

    </activity>

</application>

</manifest>
```

Note: If your project uses a different theme name, change @style/Theme.LogCatDemo to the theme already defined in your project.

How to view the output

1. Run the application.
2. Open **Logcat** in Android Studio.
3. Search for ActivityLifecycle.
4. Press Home or switch apps to observe more lifecycle messages.

Example Logcat output:

ActivityLifecycle: onCreate invoked

ActivityLifecycle: onStart invoked

ActivityLifecycle: onResume invoked

ActivityLifecycle: onPause invoked

ActivityLifecycle: onStop invoked

The exact sequence depends on how you interact with the application.

Activity Lifecycle Methods:

- onCreate() — Activity is created.
- onStart() — Activity becomes visible.
- onResume() — Activity becomes interactive.
- onPause() — Activity loses foreground focus.
- onStop() — Activity is no longer visible.

- onDestroy() — Activity is being destroyed.

PRACTICAL 2(a): CREATE AND INTEGRATE A FRAGMENT AND DEMONSTRATE FRAGMENT LIFECYCLE

Project Name: FragmentLifecycleDemo
Fragment Name: DemoFragment
Fragment Layout: fragment_demo
Template: Empty Views Activity
Language: Java

1. DemoFragment.java

```
package com.example.fragmentlifecycledemo;


import android.content.Context;

import android.os.Bundle;

import android.util.Log;

import android.view.LayoutInflater;

import android.view.View;

import android.view.ViewGroup;


import androidx.annotation.NonNull;

import androidx.annotation.Nullable;

import androidx.fragment.app.Fragment;


public class DemoFragment extends Fragment {


   String TAG = "FragmentLifecycle";


   @Override

   public void onAttach(@NonNull Context context) {

      super.onAttach(context);
```

```
    Log.d(TAG, "onAttach() executed");
}

@Override
public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    Log.d(TAG, "onCreate() executed");
}

@Override
public View onCreateView(LayoutInflater inflater, ViewGroup container,
                         Bundle savedInstanceState) {
    Log.d(TAG, "onCreateView() executed");
    return inflater.inflate(R.layout.fragment_demo, container, false);
}

@Override
public void onViewCreated(@NonNull View view,
                          @Nullable Bundle savedInstanceState) {
    super.onViewCreated(view, savedInstanceState);
    Log.d("on view created invoked", "onViewCreated() executed");
}

@Override
public void onStart() {
    super.onStart();
    Log.d("on start invoked", "on started");
}
```

```
@Override
public void onResume() {
   super.onResume();
   Log.d("on resume invoked", "on resume");
}

@Override
public void onPause() {
   super.onPause();
   Log.d("on pause invoked", "on pause");
}

@Override
public void onStop() {
   super.onStop();
   Log.d("on stop invoked", "on stopped");
}

@Override
public void onDestroyView() {
   super.onDestroyView();
   Log.d("on destroyview invoked", "on destroyview() executed");
}

@Override
public void onDestroy() {
   super.onDestroy();
```

```
        Log.d("on destroy invoked", "on destroy");
    }

    @Override
    public void onDetach() {
        super.onDetach();
        Log.d("on detach invoked", "on detach");
    }
}
```

2. activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/fragment_container_view_tag"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
```

```
</androidx.constraintlayout.widget.ConstraintLayout>
```

3. MainActivity.java

```
package com.example.fragmentlifecycledemo;

import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
import androidx.fragment.app.FragmentTransaction;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        DemoFragment fragment = new DemoFragment();

        FragmentTransaction transaction =
                getSupportFragmentManager().beginTransaction();

        transaction.replace(R.id.fragment_container_view_tag, fragment);
        transaction.commit();
    }
}
```

4. fragment_demo.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout
```

```
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".DemoFragment">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:text="@string/hello_blank_fragment" />

</FrameLayout>
```

Expected Output: The fragment is displayed in the activity. Open **Logcat** in Android Studio to observe the fragment lifecycle messages.

PRACTICAL 2(b): FRAGMENT-BASED UI NAVIGATION BETWEEN MULTIPLE SCREENS

Project Name: FragmentLifecycle
Template: Empty Views Activity
Language: Java
Files Required: 6

1. MainActivity.java

```
package com.example.fragmentlifecycle;

import androidx.appcompat.app.AppCompatActivity;
import androidx.fragment.app.FragmentManager;
import androidx.fragment.app.FragmentTransaction;

import android.os.Bundle;
import android.widget.Button;
```

```
public class MainActivity extends AppCompatActivity {

    private boolean isFragmentOneLoaded = true;
    private FragmentManager fragmentManager;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        fragmentManager = getSupportFragmentManager();

        Button btnChange = findViewById(R.id.btn_change);

        loadFragmentOne();

        btnChange.setOnClickListener(v -> {
            if (isFragmentOneLoaded) {
                loadFragmentTwo();
            } else {
                loadFragmentOne();
            }
        });
    }

    private void loadFragmentOne() {
        FragmentTransaction transaction =
```

```
            fragmentManager.beginTransaction();

        FragmentOne fragment = new FragmentOne();

        transaction.replace(R.id.fragment_holder, fragment);
        transaction.addToBackStack(null);
        transaction.commit();

        isFragmentOneLoaded = true;
    }

    private void loadFragmentTwo() {
        FragmentTransaction transaction =
                fragmentManager.beginTransaction();

        FragmentTwo fragment = new FragmentTwo();

        transaction.replace(R.id.fragment_holder, fragment);
        transaction.addToBackStack(null);
        transaction.commit();

        isFragmentOneLoaded = false;
    }
}
```

2. FragmentOne.java

```
package com.example.fragmentlifecycle;

import android.content.Context;
```

```
import android.os.Bundle;
import android.util.Log;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;

import androidx.fragment.app.Fragment;

public class FragmentOne extends Fragment {

    private static final String TAG = "FragmentOne";

    @Override
    public void onAttach(Context context) {
        super.onAttach(context);
        Log.d(TAG, "onAttach");
    }

    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup container,
                             Bundle savedInstanceState) {
        Log.d(TAG, "onCreateView");
        return inflater.inflate(R.layout.fragment_one, container, false);
    }
}
```

3. FragmentTwo.java

```
package com.example.fragmentlifecycle;
```

```
import android.content.Context;
import android.os.Bundle;
import android.util.Log;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;

import androidx.fragment.app.Fragment;

public class FragmentTwo extends Fragment {

    private static final String TAG = "FragmentTwo";

    @Override
    public void onAttach(Context context) {
        super.onAttach(context);
        Log.d(TAG, "onAttach");
    }

    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup container,
                             Bundle savedInstanceState) {
        Log.d(TAG, "onCreateView");
        return inflater.inflate(R.layout.fragment_two, container, false);
    }
}
```

4. activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <Button
        android:id="@+id/btn_change"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_margin="15dp"
        android:backgroundTint="@android:color/holo_green_dark"
        android:text="Change Fragment"
        android:textAllCaps="false"
        android:textSize="24sp" />

    <FrameLayout
        android:id="@+id/fragment_holder"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1" />

</LinearLayout>
```

5. fragment_one.xml

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
```

```
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_centerInParent="true"
        android:text="Fragment One"
        android:textSize="24sp" />

</RelativeLayout>
```

6. fragment_two.xml

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/textView2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_centerInParent="true"
        android:text="Fragment Two"
        android:textSize="24sp" />
```

</RelativeLayout>

Expected Output: Fragment One appears first. Clicking **Change Fragment** switches to Fragment Two. Clicking it again switches back to Fragment One.

PRACTICAL 2(c): COMMUNICATION BETWEEN ACTIVITY AND FRAGMENT

Project Name: ActivityFragmentCommunication
Fragment Name: DemoFragment
Fragment Layout: fragment_demo
Template: Empty Views Activity
Language: Java

1. activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>

<FrameLayout

   xmlns:android="http://schemas.android.com/apk/res/android"

   android:id="@+id/fragment_container"

   android:layout_width="match_parent"

   android:layout_height="match_parent" />
```

2. fragment_demo.xml

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout

   xmlns:android="http://schemas.android.com/apk/res/android"

   android:layout_width="match_parent"

   android:layout_height="match_parent"

   android:gravity="center"

   android:orientation="vertical"

   android:padding="20dp">


   <TextView

      android:id="@+id/txtMessage"
```

```
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="22sp"
        android:text="Message from Activity" />

    <Button
        android:id="@+id/btnSend"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Send Message to Activity"
        android:layout_marginTop="20dp" />

</LinearLayout>
```

3. DemoFragment.java

```
package com.example.activityfragmentcommunication;

import android.content.Context;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
```

```
public class DemoFragment extends Fragment {

    TextView txtMessage;
    Button btnSend;
    String activityMessage;

    FragmentCommunication communication;

    public interface FragmentCommunication {
        void onMessageFromFragment(String message);
    }

    // Receive Activity object
    @Override
    public void onAttach(@NonNull Context context) {
        super.onAttach(context);
        communication = (FragmentCommunication) context;
    }

    // Method to receive data from Activity
    public static DemoFragment newInstance(String message) {
        DemoFragment fragment = new DemoFragment();

        Bundle bundle = new Bundle();
        bundle.putString("msg", message);

        fragment.setArguments(bundle);
```

```
        return fragment;
    }

    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup container,
                             Bundle savedInstanceState) {

        View view = inflater.inflate(
                R.layout.fragment_demo, container, false);

        txtMessage = view.findViewById(R.id.txtMessage);
        btnSend = view.findViewById(R.id.btnSend);

        // Receiving data from Activity
        if (getArguments() != null) {
            activityMessage = getArguments().getString("msg");
            txtMessage.setText(activityMessage);
        }

        // Sending data to Activity
        btnSend.setOnClickListener(v -> {
            communication.onMessageFromFragment(
                    "Hello Activity! Message from Fragment");
        });

        return view;
    }
}
```

4. MainActivity.java

```
package com.example.activityfragmentcommunication;

import android.os.Bundle;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity
        implements DemoFragment.FragmentCommunication {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        // Sending data from Activity to Fragment
        DemoFragment fragment = DemoFragment.newInstance(
                "Hello Fragment! Message from Activity"
        );

        getSupportFragmentManager()
                .beginTransaction()
                .replace(R.id.fragment_container, fragment)
                .commit();
    }

    // Receiving data from Fragment
```

```
    @Override
    public void onMessageFromFragment(String message) {
        Toast.makeText(this, message, Toast.LENGTH_LONG).show();
    }
}
```

Expected Output:

- The Fragment displays: **Hello Fragment! Message from Activity**
- Clicking **Send Message to Activity** displays a Toast message: **Hello Activity! Message from Fragment**

PRACTICAL 3(a): LINEARLAYOUT — HORIZONTAL AND VERTICAL

Aim: Develop an Android application using LinearLayout (Horizontal and Vertical) to design a structured user interface.

Project Name: LinearLayout
Template: Empty Views Activity
Language: Java

1. Filename: activity_main.xml

Location: app/src/main/res/layout/activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="20dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
```

```
    android:text="Student Registration"
    android:textSize="26sp"
    android:textStyle="bold"
    android:layout_gravity="center"
    android:layout_marginBottom="20dp"/>

<EditText
    android:id="@+id/etName"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter Name"
    android:layout_marginBottom="10dp"/>

<EditText
    android:id="@+id/etEmail"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter Email"
    android:inputType="textEmailAddress"
    android:layout_marginBottom="20dp"/>

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:gravity="center">

    <Button
```

```
        android:id="@+id/btnSubmit"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Submit"/>

    <Button
        android:id="@+id/btnReset"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Reset"
        android:layout_marginStart="20dp"/>

  </LinearLayout>

</LinearLayout>
```

2. Filename: MainActivity.java

Location: app/src/main/java/com/example/linearlayout/MainActivity.java

```
package com.example.linearlayout;

import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
```

```
EditText etName, etEmail;
Button btnSubmit, btnReset;

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    etName = findViewById(R.id.etName);
    etEmail = findViewById(R.id.etEmail);
    btnSubmit = findViewById(R.id.btnSubmit);
    btnReset = findViewById(R.id.btnReset);

    btnSubmit.setOnClickListener(v -> {
        String name = etName.getText().toString();
        String email = etEmail.getText().toString();

        Toast.makeText(
                this,
                "Name: " + name + "\nEmail: " + email,
                Toast.LENGTH_LONG
        ).show();
    });

    btnReset.setOnClickListener(v -> {
        etName.setText("");
        etEmail.setText("");
    });
```

```
    }
}
```

Expected Output

A Student Registration form containing:

- Student Registration heading
- Enter Name field
- Enter Email field
- Submit button
- Reset button

Working: The fields are arranged vertically. The Submit and Reset buttons are arranged horizontally. Submit displays the entered name and email in a Toast message, while Reset clears both fields.

PRACTICAL 3(b): RELATIVELAYOUT — DYNAMIC ALIGNMENT

Aim: Develop an Android application using RelativeLayout to align UI components dynamically.

Project Name: RelativeLayout
Template: Empty Views Activity
Language: Java

1. Filename: activity_main.xml

Location: app/src/main/res/layout/activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
```

```
android:padding="16dp"
tools:context=".MainActivity">

<TextView
    android:id="@+id/titleText"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_centerHorizontal="true"
    android:layout_marginTop="20dp"
    android:text="RelativeLayout Demo"
    android:textAlignment="center"
    android:textSize="20sp"
    android:textStyle="bold"/>

<ImageView
    android:id="@+id/imageView6"
    android:layout_width="268dp"
    android:layout_height="126dp"
    android:layout_below="@id/titleText"
    android:layout_centerHorizontal="true"
    android:layout_marginTop="20dp"
    app:srcCompat="@drawable/zoro"/>

<EditText
    android:id="@+id/editTextText2"
    android:layout_width="match_parent"
    android:layout_height="64dp"
    android:layout_below="@id/imageView6"
```

```
    android:layout_marginTop="20dp"
    android:ems="10"
    android:hint="Project Name"
    android:inputType="text"/>

<Button
    android:id="@+id/button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_below="@id/editTextText2"
    android:layout_alignParentStart="true"
    android:layout_marginTop="20dp"
    android:text="Sumbit"/>

<Button
    android:id="@+id/button2"
    android:layout_width="95dp"
    android:layout_height="52dp"
    android:layout_below="@id/editTextText2"
    android:layout_alignParentEnd="true"
    android:layout_marginTop="20dp"
    android:text="Reset"/>

</RelativeLayout>
```

Required Image

The XML uses the image resource @drawable/zoro.

1. Get the image you want to use.
2. Rename it to zoro.png or zoro.jpg.

3. Place it inside:

app/src/main/res/drawable/

The filename must be lowercase, and the extension must match the actual image file.

Expected Output

The application displays:

- A heading reading **RelativeLayout Demo**
- An image below the heading
- A Project Name input field below the image
- Submit and Reset buttons positioned on opposite sides

Note: The PDF provides the XML layout for this practical, not Java code for the button actions. Therefore, the Submit and Reset buttons are displayed but do not perform actions in this version.

PRACTICAL 3(c): TABLELAYOUT — LOGIN/REGISTRATION FORM

Aim: Develop an Android application using TableLayout to design a structured Login/Registration Form.

Project Name: TableLayout
Template: Empty Views Activity
Language: Java

1. Filename: activity_main.xml

Location: app/src/main/res/layout/activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
```

```
android:fillViewport="true"
tools:context=".MainActivity">

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="24dp">

    <TextView
        android:id="@+id/tvHeader"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginBottom="32dp"
        android:text="Registration Form"
        android:textColor="@android:color/black"
        android:textSize="28sp"
        android:textStyle="bold"/>

    <TableLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:stretchColumns="1">

        <!-- Name Row -->
        <TableRow
            android:layout_width="match_parent"
            android:layout_height="wrap_content">
```

```
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center_vertical"
        android:paddingEnd="12dp"
        android:text="Name:"
        android:textColor="@android:color/black"
        android:textSize="16sp"
        android:textStyle="bold"/>

    <EditText
        android:id="@+id/etName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter Name"
        android:inputType="textPersonName"/>

</TableRow>

<!-- Email Row -->
<TableRow
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp">

    <TextView
        android:layout_width="wrap_content"
```

```
        android:layout_height="wrap_content"
        android:layout_gravity="center_vertical"
        android:paddingEnd="12dp"
        android:text="Email:"
        android:textColor="@android:color/black"
        android:textSize="16sp"
        android:textStyle="bold"/>

    <EditText
        android:id="@+id/etEmail"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter Email"
        android:inputType="textEmailAddress"/>

</TableRow>

<!-- Password Row -->
<TableRow
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center_vertical"
        android:paddingEnd="12dp"
```

```
        android:text="Password:"
        android:textColor="@android:color/black"
        android:textSize="16sp"
        android:textStyle="bold"/>

    <EditText
        android:id="@+id/etPassword"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter Password"
        android:inputType="textPassword"/>

</TableRow>

<!-- Buttons Row -->
<TableRow
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="24dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_span="2"
        android:orientation="horizontal">

        <Button
            android:id="@+id/btnRegister"
```

```
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_marginEnd="8dp"
            android:layout_weight="1"
            android:backgroundTint="#3F51B5"
            android:text="Register"
            android:textAllCaps="false"
            android:textColor="@android:color/white"
            app:cornerRadius="20dp"/>

        <Button
            android:id="@+id/btnReset"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_marginStart="8dp"
            android:layout_weight="1"
            android:backgroundTint="#3F51B5"
            android:text="Reset"
            android:textAllCaps="false"
            android:textColor="@android:color/white"
            app:cornerRadius="20dp"/>

    </LinearLayout>

  </TableRow>

</TableLayout>
```

```
    </LinearLayout>


</ScrollView>
```

Expected Output

The application displays a Registration Form containing:

- Name field
- Email field
- Password field
- Register button
- Reset button

The labels and input fields are arranged in rows and columns using TableLayout. The form is placed inside a ScrollView so the content can scroll on smaller screens.

Note: The PDF provides only the XML layout for this practical. The Register and Reset buttons will appear, but their functionality requires Java code, which is not included in the source practical.

PRACTICAL 4(a): ScrollView — Long Scrollable Content

Project Name: ScrollView
Filename: activity_main.xml

Code: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>


<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
```

```
tools:context=".MainActivity">

<ScrollView
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:ignore="MissingConstraints">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical">

            <TextView
                android:id="@+id/textView2"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:paddingTop="10dp"
                android:text="Student Registration Form"
                android:textAlignment="center"
                android:textSize="24sp"
                android:textStyle="bold" />

            <EditText
```

```
    android:id="@+id/editTextText"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:ems="10"
    android:inputType="text"
    android:hint="Enter Full Name"
    android:paddingTop="25dp" />

<EditText
    android:id="@+id/editTextText2"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:ems="10"
    android:inputType="textEmailAddress"
    android:hint="Enter Email"
    android:paddingTop="25dp" />

<EditText
    android:id="@+id/editTextText3"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:ems="10"
    android:inputType="phone"
    android:hint="Enter Mobile Number"
    android:paddingTop="25dp" />

<EditText
    android:id="@+id/editTextText4"
```

```
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:ems="10"
    android:inputType="textPostalAddress"
    android:hint="Enter Address"
    android:paddingTop="25dp" />

<EditText
    android:id="@+id/editTextText5"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:ems="10"
    android:inputType="text"
    android:hint="Enter City"
    android:paddingTop="25dp" />

<EditText
    android:id="@+id/editTextText6"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:ems="10"
    android:inputType="text"
    android:hint="Enter State"
    android:paddingTop="25dp" />

<EditText
    android:id="@+id/editTextText7"
    android:layout_width="match_parent"
```

```
    android:layout_height="wrap_content"
    android:ems="10"
    android:inputType="text"
    android:hint="Enter Country"
    android:paddingTop="25dp" />

<EditText
    android:id="@+id/editTextText8"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:ems="10"
    android:inputType="text"
    android:hint="Enter Qualification"
    android:paddingTop="25dp" />

<EditText
    android:id="@+id/editTextText9"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:ems="10"
    android:inputType="numberDecimal"
    android:hint="X Percentage"
    android:paddingTop="25dp" />

<EditText
    android:id="@+id/editTextText10"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
```

```
    android:ems="10"
    android:inputType="text"
    android:hint="Enter Experience"
    android:paddingTop="25dp" />

<EditText
    android:id="@+id/editTextText11"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:ems="10"
    android:inputType="text"
    android:hint="Enter Country"
    android:paddingTop="25dp" />

<EditText
    android:id="@+id/editTextText12"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:ems="10"
    android:inputType="numberDecimal"
    android:hint="XII Percentage"
    android:paddingTop="25dp" />

<EditText
    android:id="@+id/editTextText13"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:ems="10"
```

```
    android:inputType="text"
    android:hint="School Name"
    android:paddingTop="25dp" />

<EditText
    android:id="@+id/editTextText14"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:ems="10"
    android:inputType="text"
    android:hint="College Name"
    android:paddingTop="25dp" />

<EditText
    android:id="@+id/editTextText15"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:ems="10"
    android:inputType="text"
    android:hint="Father Name"
    android:paddingTop="25dp" />

<Button
    android:id="@+id/button"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Submit" />
```

```
        <Button
            android:id="@+id/button2"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Reset" />

    </LinearLayout>

  </LinearLayout>

</ScrollView>

</androidx.constraintlayout.widget.ConstraintLayout>
```

Note: This XML creates the long student registration form. The ScrollView allows the user to scroll down to fields that do not fit on the screen. The Submit and Reset buttons are displayed, but their actions are not programmed in the PDF.

PRACTICAL 4(b): AbsoluteLayout — Fixed Positioning

Project Name: AbsoluteLayout
Filename: activity_main.xml

Code: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>

<AbsoluteLayout
  xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:app="http://schemas.android.com/apk/res-auto"
  xmlns:tools="http://schemas.android.com/tools"
  android:id="@+id/main"
  android:layout_width="match_parent"
```

```
android:layout_height="match_parent"
android:rotationX="0"
tools:context=".MainActivity">

<TextView
   android:id="@+id/textView4"
   android:layout_width="wrap_content"
   android:layout_height="wrap_content"
   android:layout_x="60dp"
   android:layout_y="30dp"
   android:text="AbsoluteLayout Demo"
   android:textSize="24sp"
   android:textStyle="bold" />

<TextView
   android:id="@+id/textView5"
   android:layout_width="wrap_content"
   android:layout_height="wrap_content"
   android:layout_x="30dp"
   android:layout_y="100dp"
   android:text="Name:"
   android:textSize="20sp" />

<EditText
   android:id="@+id/editTextText"
   android:layout_width="wrap_content"
   android:layout_height="wrap_content"
   android:layout_x="100dp"
```

```
    android:layout_y="90dp"
    android:ems="10"
    android:hint="Enter Name"
    android:inputType="text" />

<TextView
    android:id="@+id/textView6"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_x="30dp"
    android:layout_y="170dp"
    android:text="Email:"
    android:textSize="20sp" />

<EditText
    android:id="@+id/editTextText2"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_x="100dp"
    android:layout_y="160dp"
    android:ems="10"
    android:hint="Enter Email"
    android:inputType="textEmailAddress" />

<Button
    android:id="@+id/button"
    android:layout_width="132dp"
    android:layout_height="wrap_content"
```

```
    android:layout_x="50dp"
    android:layout_y="250dp"
    android:text="Submit" />

  <Button
    android:id="@+id/button2"
    android:layout_width="150dp"
    android:layout_height="wrap_content"
    android:layout_x="190dp"
    android:layout_y="250dp"
    android:text="Reset" />

</AbsoluteLayout>
```

Note: This layout demonstrates fixed positioning using android:layout_x and android:layout_y. These attributes specify where each component appears on the screen.

Important: AbsoluteLayout is deprecated in modern Android development, but this example follows the practical's requirement. The Submit and Reset buttons do not perform any action unless Java code is added.

PRACTICAL 4(c): FrameLayout — Overlay UI Elements

Project Name: FrameLayout
Filename: activity_main.xml

Code: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>

<androidx.constraintlayout.widget.ConstraintLayout
  xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:app="http://schemas.android.com/apk/res-auto"
  xmlns:tools="http://schemas.android.com/tools"
```

```
android:id="@+id/main"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".MainActivity">

<FrameLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:ignore="MissingConstraints">

    <ImageView
        android:id="@+id/imageView"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:scaleType="centerCrop"
        app:srcCompat="@drawable/baldski" />

    <Button
        android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="bottom|center_horizontal"
        android:layout_marginBottom="20dp"
        android:text="Click Here" />

    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
```

```
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:paddingLeft="30dp"
        android:text="Welcome to FrameLayout"
        android:textColor="#FFFFFF"
        android:textSize="30sp" />

    </FrameLayout>

</androidx.constraintlayout.widget.ConstraintLayout>
```

Important for P4(c)

1. The practical uses an image named **baldski**.
2. Put the image file inside app/src/main/res/drawable/.
3. The filename should be baldski.png, baldski.jpg, or another supported image format, and the XML reference must match the filename without its extension.
4. If you do not have the image, use your own image and change @drawable/baldski to the name of your image resource.
5. The FrameLayout places the image at the back, with the text and button layered over it.

Note: The button is visible but does not perform an action because the PDF does not provide Java code for its click event.

Quick Revision for the Practical Exam

Practical	Layout used	Main purpose
P4(a)	ScrollView	Displays long content that can be scrolled vertically.
P4(b)	AbsoluteLayout	Places UI components at fixed X and Y coordinates.
P4(c)	FrameLayout	Places UI elements on top of one another.

Remember:

- **ScrollView = Scrolling**

- **AbsoluteLayout = Fixed position**
- **FrameLayout = Overlay**

Android Studio: Create a separate project for each practical and replace the contents of its activity_main.xml file with the corresponding code above. If Android Studio reports an error involving AbsoluteLayout, remember that it is deprecated; the practical specifically asks you to demonstrate it.

PRACTICAL 5(a): ListView with ArrayAdapter and Item Click Handling

Project Name: ListViewDemo

1. Filename: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>


<LinearLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   android:id="@+id/main"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:orientation="vertical"
   android:padding="10dp">


   <TextView
      android:id="@+id/textView"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:paddingBottom="10dp"
      android:text="Programming Language"
      android:textSize="24sp"
      android:textStyle="bold" />


   <ListView
```

```
        android:id="@+id/listView"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1" />

</LinearLayout>
```

2. Filename: MainActivity.java

```
package com.example.listviewdemo;

import android.os.Bundle;
import android.view.View;
import android.widget.AdapterView;
import android.widget.ArrayAdapter;
import android.widget.ListView;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    ListView listView;

    String[] languages = {
            "Java",
            "Python",
            "C",
            "C++",
            "Kotlin",
```

```
        "PHP",
        "Javascript",
        "Swift"
};

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    listView = findViewById(R.id.listView);

    ArrayAdapter<String> adapter = new ArrayAdapter<>(
            this,
            android.R.layout.simple_list_item_1,
            languages
    );

    listView.setAdapter(adapter);

    listView.setOnItemClickListener(
            new AdapterView.OnItemClickListener() {
                @Override
                public void onItemClick(
                        AdapterView<?> parent,
                        View view,
                        int position,
                        long id
```

```
            ) {
                String item = languages[position];


                Toast.makeText(
                        MainActivity.this,
                        "Selected: " + item,
                        Toast.LENGTH_SHORT
                ).show();
            }
        }
    );
  }
}
```

Output

- A heading displays **Programming Language**.
- The list displays Java, Python, C, C++, Kotlin, PHP, Javascript and Swift.
- When you tap a language, a Toast message displays the selected language.

Remember

- **ListView:** Displays items in a vertical list.
- **ArrayAdapter:** Connects an array of data to the ListView.
- **OnItemClickListener:** Detects when the user taps an item.
- **Toast:** Displays a short message on the screen.

PRACTICAL 5(b): GridView with Custom Adapter and Image Selection

Project Name: GridViewDemo

This practical displays images in a two-column grid. Each image has a name, and tapping an image displays its selected name.

1. Filename: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="10dp">

    <TextView
        android:id="@+id/textView2"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:paddingBottom="10dp"
        android:text="Image Gallery"
        android:textSize="24sp"
        android:textStyle="bold" />

    <GridView
        android:id="@+id/gridView"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:numColumns="2"
        android:verticalSpacing="10dp"
        android:horizontalSpacing="10dp"
        android:stretchMode="columnWidth" />
```

```
</LinearLayout>
```

2. Filename: grid_item.xml

Create this file inside **app → res → layout**.

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="10dp">

    <ImageView
        android:id="@+id/imageView"
        android:layout_width="120dp"
        android:layout_height="120dp"
        android:scaleType="centerCrop"
        android:contentDescription="Gallery image" />

    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="18sp"
        android:paddingTop="5dp" />
```

```
</LinearLayout>
```

3. Filename: CustomAdapter.java

```
package com.example.gridviewdemo;

import android.content.Context;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.BaseAdapter;
import android.widget.ImageView;
import android.widget.TextView;

public class CustomAdapter extends BaseAdapter {

    Context context;
    String[] imageNames;
    int[] images;
    LayoutInflater inflater;

    public CustomAdapter(
            Context context,
            String[] imageNames,
            int[] images
    ) {
        this.context = context;
        this.imageNames = imageNames;
        this.images = images;
        inflater = LayoutInflater.from(context);
```

```
}

@Override
public int getCount() {
    return imageNames.length;
}

@Override
public Object getItem(int position) {
    return imageNames[position];
}

@Override
public long getItemId(int position) {
    return position;
}

@Override
public View getView(
        int position,
        View convertView,
        ViewGroup parent
) {
    View view = inflater.inflate(
            R.layout.grid_item,
            parent,
            false
    );
```

```
        ImageView imageView =
                view.findViewById(R.id.imageView);

        TextView textView =
                view.findViewById(R.id.textView);

        imageView.setImageResource(images[position]);
        textView.setText(imageNames[position]);

        return view;
    }
}
```

4. Filename: MainActivity.java

```
package com.example.gridviewdemo;

import android.os.Bundle;
import android.view.View;
import android.widget.AdapterView;
import android.widget.GridView;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    GridView gridView;
```

```
String[] imageNames = {
        "Human",
        "Food",
        "Tree",
        "Animal"
};

int[] images = {
        R.drawable.image1,
        R.drawable.image2,
        R.drawable.image3,
        R.drawable.image4
};

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    gridView = findViewById(R.id.gridView);

    CustomAdapter adapter = new CustomAdapter(
            this,
            imageNames,
            images
    );

    gridView.setAdapter(adapter);
```

```
        gridView.setOnItemClickListener(
                new AdapterView.OnItemClickListener() {
                    @Override
                    public void onItemClick(
                            AdapterView<?> parent,
                            View view,
                            int position,
                            long id
                    ) {
                        Toast.makeText(
                                MainActivity.this,
                                "Selected: " + imageNames[position],
                                Toast.LENGTH_SHORT
                        ).show();
                    }
                }
        );
    }
}
```

5. Required images

Create or use four image files and place them inside:

app/src/main/res/drawable/

Use these exact filenames:

- image1.png
- image2.png
- image3.png
- image4.png

You can use JPG images instead, but update the references only if you change the resource names. Android resource names must use lowercase letters, numbers and underscores.

Important: If you do not add these images, Android Studio will report an error for R.drawable.image1, R.drawable.image2, R.drawable.image3 and R.drawable.image4.

Output

- The screen displays **Image Gallery**.
- Four images appear in a two-column grid.
- Each image has a label: Human, Food, Tree or Animal.
- Tapping an image displays a Toast with its name.

Remember

- **GridView:** Displays items in a grid.
- **CustomAdapter:** Controls how each grid item looks.
- **BaseAdapter:** Provides the basic adapter functionality.
- **getView():** Creates the view for each grid item.
- **setOnItemClickListener():** Handles image selection.

PRACTICAL 5(c): RecyclerView — Modern Alternative to ListView

Project Name: RecyclerViewDemo

This practical displays programming languages using RecyclerView and a custom adapter.

1. Filename: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>


<LinearLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   android:id="@+id/main"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:orientation="vertical">
```

```
    <TextView
        android:id="@+id/textView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginVertical="30dp"
        android:text="RecyclerViewDemo"
        android:textAlignment="center"
        android:textSize="20sp"
        android:paddingBottom="10dp"
        android:textStyle="bold" />

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/recyclerView"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1" />

</LinearLayout>
```

2. Filename: item_layout.xml

Create this file inside **app → res → layout**.

```
<?xml version="1.0" encoding="utf-8"?>

<TextView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/txtItem"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="20sp"
```

```
   android:padding="15dp" />
```

3. Filename: MyAdapter.java

```
package com.example.recyclerviewdemo;

import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

public class MyAdapter
      extends RecyclerView.Adapter<MyAdapter.ViewHolder> {

   String[] data;

   public MyAdapter(String[] data) {
      this.data = data;
   }

   public static class ViewHolder
         extends RecyclerView.ViewHolder {

      TextView txtItem;

      public ViewHolder(@NonNull View itemView) {
```

```
        super(itemView);

        txtItem = itemView.findViewById(R.id.txtItem);
    }
}

@NonNull
@Override
public ViewHolder onCreateViewHolder(
        @NonNull ViewGroup parent,
        int viewType
) {
    View view = LayoutInflater.from(parent.getContext())
            .inflate(
                    R.layout.item_layout,
                    parent,
                    false
            );

    return new ViewHolder(view);
}

@Override
public void onBindViewHolder(
        @NonNull ViewHolder holder,
        int position
) {
    holder.txtItem.setText(data[position]);
```

```
        holder.itemView.setOnClickListener(v -> {
            Toast.makeText(
                    v.getContext(),
                    "Selected: " + data[position],
                    Toast.LENGTH_SHORT
            ).show();
        });
    }

    @Override
    public int getItemCount() {
        return data.length;
    }
}
```

4. Filename: MainActivity.java

```
package com.example.recyclerviewdemo;

import android.os.Bundle;

import androidx.appcompat.app.AppCompatActivity;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

public class MainActivity extends AppCompatActivity {

    RecyclerView recyclerView;
```

```
String[] languages = {
        "Java",
        "Python",
        "C",
        "C++",
        "Kotlin",
        "PHP",
        "JavaScript",
        "Swift",
        "Go",
        "Ruby"
};

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    recyclerView = findViewById(R.id.recyclerView);

    recyclerView.setLayoutManager(
            new LinearLayoutManager(this)
    );

    MyAdapter adapter = new MyAdapter(languages);

    recyclerView.setAdapter(adapter);
}
```

}

5. RecyclerView dependency

Open **Gradle Scripts → build.gradle (Module: app)**.

Inside the dependencies block, add:

implementation("androidx.recyclerview:recyclerview:1.3.2")

For example:

```
dependencies {
    implementation(libs.appcompat)
    implementation(libs.material)
    implementation(libs.activity)
    implementation(libs.constraintlayout)


    implementation("androidx.recyclerview:recyclerview:1.3.2")
}
```

If the dependency is already present, do not add it a second time. Click **Sync Now** after making changes.

Output

- The screen displays **RecyclerViewDemo**.
- A vertical list of programming languages appears.
- Tapping a language displays a Toast with the selected language.

Remember

- **RecyclerView:** Displays a scrollable list of items efficiently.
- **Adapter:** Connects data to RecyclerView.
- **ViewHolder:** Holds references to the views used for each item.
- **onCreateViewHolder():** Creates the item view.
- **onBindViewHolder():** Places data into the item view.
- **getItemCount():** Returns the number of items.
- **LinearLayoutManager:** Displays the items in a vertical or horizontal list.

PRACTICAL 6(a): AppBar (Toolbar) and Options Menu

Project Name: ToolbarMenuDemo

1. Filename: activity_main.xml

Place this file inside app → res → layout → activity_main.xml.

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   xmlns:app="http://schemas.android.com/apk/res-auto"
   android:id="@+id/main"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:orientation="vertical">

   <androidx.appcompat.widget.Toolbar
      android:id="@+id/toolbar"
      android:layout_width="match_parent"
      android:layout_height="?attr/actionBarSize"
      android:background="?attr/colorPrimary"
      android:theme="@style/ThemeOverlay.AppCompat.Dark.ActionBar"
      app:title="Toolbar Demo"
      app:titleTextColor="@android:color/white" />

   <TextView
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="AppBar and Options Menu Example"
```

```
    android:textSize="20sp"
    android:padding="22dp" />

</LinearLayout>
```

2. Filename: menu_main.xml

Create a menu folder inside app → res if it does not already exist. Create menu_main.xml inside that folder.

```
<?xml version="1.0" encoding="utf-8"?>

<menu xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:app="http://schemas.android.com/apk/res-auto">

  <item
    android:id="@+id/menu_home"
    android:title="Home"
    app:showAsAction="never" />

  <item
    android:id="@+id/menu_about"
    android:title="About"
    app:showAsAction="never" />

  <item
    android:id="@+id/menu_logout"
    android:title="Logout"
    app:showAsAction="never" />

  <item
```

```
        android:id="@+id/menu_settings"
        android:title="Settings"
        app:showAsAction="never" />

</menu>
```

3. Filename: MainActivity.java

```
package com.example.toolbarmenudemo;

import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.appcompat.widget.Toolbar;

public class MainActivity extends AppCompatActivity {

    Toolbar toolbar;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        toolbar = findViewById(R.id.toolbar);
        setSupportActionBar(toolbar);
```

```
}

@Override
public boolean onCreateOptionsMenu(Menu menu) {
   getMenuInflater().inflate(R.menu.menu_main, menu);
   return true;
}

@Override
public boolean onOptionsItemSelected(@NonNull MenuItem item) {

   int id = item.getItemId();

   if (id == R.id.menu_home) {
      Toast.makeText(this, "Home Selected",
            Toast.LENGTH_SHORT).show();

   } else if (id == R.id.menu_settings) {
      Toast.makeText(this, "Settings Selected",
            Toast.LENGTH_SHORT).show();

   } else if (id == R.id.menu_about) {
      Toast.makeText(this, "About Selected",
            Toast.LENGTH_SHORT).show();

   } else if (id == R.id.menu_logout) {
      Toast.makeText(this, "Logout Selected",
            Toast.LENGTH_SHORT).show();
```

```
        }


        return true;
    }
}
```

Output

- A Toolbar displays the title **Toolbar Demo**.
- The options menu contains Home, About, Logout and Settings.
- Selecting an item displays a Toast message.

Remember:

- Toolbar displays the app's top bar.
- onCreateOptionsMenu() creates the menu.
- onOptionsItemSelected() handles menu selections.
- Toast displays a short message.

PRACTICAL 6(b): AlertDialog and Custom DialogFragment

Project Name: DialogDemo

This practical demonstrates a confirmation dialog and a custom dialog containing input fields.

1. Filename: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>


<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
```

```
    android:padding="20dp">

    <Button
        android:id="@+id/btnAlert"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Show AlertDialog" />

    <Button
        android:id="@+id/btnCustom"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Show Custom Dialog"
        android:layout_marginTop="20dp" />

</LinearLayout>
```

Note: The PDF's Java code references btnAlert and btnCustom, but its displayed XML uses different IDs. The XML above corrects this mismatch so the program can run.

2. Filename: dialog_custom.xml

Create this file inside app → res → layout.

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="20dp">
```

```
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Login"
        android:textSize="22sp"
        android:textStyle="bold" />

    <EditText
        android:id="@+id/etName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter User Name"
        android:inputType="text"
        android:layout_marginTop="15dp" />

    <EditText
        android:id="@+id/etName1"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter Password"
        android:inputType="textPassword"
        android:layout_marginTop="15dp" />

</LinearLayout>
```

3. Filename: CustomDialogFragment.java

```
package com.example.dialogdemo;
```

```
import android.app.Dialog;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.widget.EditText;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AlertDialog;
import androidx.fragment.app.DialogFragment;

public class CustomDialogFragment extends DialogFragment {

    EditText etName;

    @NonNull
    @Override
    public Dialog onCreateDialog(Bundle savedInstanceState) {

        AlertDialog.Builder builder =
                new AlertDialog.Builder(requireActivity());

        LayoutInflater inflater =
                requireActivity().getLayoutInflater();

        View view = inflater.inflate(R.layout.dialog_custom, null);

        etName = view.findViewById(R.id.etName);
```

```
        builder.setView(view)
                .setTitle("Input Dialog")
                .setPositiveButton("Submit", (dialog, which) -> {

                    String name = etName.getText().toString();

                    Toast.makeText(
                            requireActivity(),
                            "Entered: " + name,
                            Toast.LENGTH_SHORT
                    ).show();

                })
                .setNegativeButton("Cancel",
                        (dialog, which) -> dialog.dismiss());

        return builder.create();
    }
}
```

4. Filename: MainActivity.java

```
package com.example.dialogdemo;

import android.os.Bundle;
import android.widget.Button;
import android.widget.Toast;

import androidx.appcompat.app.AlertDialog;
```

```
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    Button btnAlert, btnCustom;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        btnAlert = findViewById(R.id.btnAlert);
        btnCustom = findViewById(R.id.btnCustom);

        // AlertDialog
        btnAlert.setOnClickListener(v -> {

            AlertDialog.Builder builder =
                    new AlertDialog.Builder(this);

            builder.setTitle("Delete File")
                    .setMessage("Are you sure you want to delete?")

                    .setPositiveButton("YES", (dialog, which) -> {

                        Toast.makeText(
                                this,
                                "File Deleted",
```

```
                    Toast.LENGTH_SHORT
                ).show();

            })

            .setNegativeButton("NO",
                (dialog, which) -> dialog.dismiss());

        builder.show();
    });

    // Custom DialogFragment
    btnCustom.setOnClickListener(v -> {

        CustomDialogFragment dialog =
            new CustomDialogFragment();

        dialog.show(
            getSupportFragmentManager(),
            "CustomDialog"
        );
    });
  }
}
```

Output

- **Show AlertDialog:** Opens a confirmation dialog with YES and NO.
- Selecting YES displays File Deleted.
- **Show Custom Dialog:** Opens a dialog containing username and password fields.

- Selecting Submit displays the entered username.

Remember:

- AlertDialog displays a message or confirmation.
- DialogFragment displays a dialog using a Fragment.
- setPositiveButton() defines the positive action.
- setNegativeButton() defines the cancel action.

PRACTICAL 6(c): Contextual Menu and Popup Menu

Project Name: MenuDemo

1. Filename: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>


<LinearLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:orientation="vertical"
   android:gravity="center"
   android:padding="20dp">


   <TextView
      android:id="@+id/txtContext"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Long Press Me"
      android:textSize="24sp"
      android:padding="20dp" />
```

```
    <Button
        android:id="@+id/btnPopup"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Show Popup Menu"
        android:layout_marginTop="30dp" />

</LinearLayout>
```

2. Filename: menu_options.xml

Create this file inside app → res → menu.

```
<?xml version="1.0" encoding="utf-8"?>

<menu xmlns:android="http://schemas.android.com/apk/res/android">

    <item
        android:id="@+id/menu_edit"
        android:title="Edit" />

    <item
        android:id="@+id/menu_delete"
        android:title="Delete" />

    <item
        android:id="@+id/menu_share"
        android:title="Share" />

</menu>
```

3. Filename: MainActivity.java

```
package com.example.menudemo;

import android.os.Bundle;
import android.view.ContextMenu;
import android.view.MenuItem;
import android.view.View;
import android.widget.Button;
import android.widget.PopupMenu;
import android.widget.TextView;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    TextView txtContext;
    Button btnPopup;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        txtContext = findViewById(R.id.txtContext);
        btnPopup = findViewById(R.id.btnPopup);

        // Register Context Menu
```

```
    registerForContextMenu(txtContext);

    // Popup Menu
    btnPopup.setOnClickListener(v -> {

        PopupMenu popupMenu =
                new PopupMenu(MainActivity.this, btnPopup);

        popupMenu.getMenuInflater().inflate(
                R.menu.menu_options,
                popupMenu.getMenu()
        );

        popupMenu.setOnMenuItemClickListener(item -> {

            Toast.makeText(
                    MainActivity.this,
                    item.getTitle(),
                    Toast.LENGTH_SHORT
            ).show();

            return true;
        });

        popupMenu.show();
    });
}
```

```
// Context Menu Creation
@Override
public void onCreateContextMenu(
        ContextMenu menu,
        View v,
        ContextMenu.ContextMenuInfo menuInfo
) {
    super.onCreateContextMenu(menu, v, menuInfo);

    menu.setHeaderTitle("Select Option");

    getMenuInflater().inflate(
            R.menu.menu_options,
            menu
    );
}

// Context Menu Click Handling
@Override
public boolean onContextItemSelected(
        @NonNull MenuItem item
) {
    Toast.makeText(
            this,
            item.getTitle(),
            Toast.LENGTH_SHORT
    ).show();
```

```
        return true;
    }
}
```

Output

- Long-press **Long Press Me** to open the contextual menu.
- Tap **Show Popup Menu** to open the popup menu.
- Both menus contain Edit, Delete and Share.
- Selecting an option displays its name in a Toast.

Remember:

- **Contextual Menu:** Usually appears after a long press on a view.
- **Popup Menu:** Appears anchored to a particular view, such as a button.
- registerForContextMenu() registers a view for a contextual menu.
- PopupMenu creates a menu attached to a view.

PRACTICAL 7(a): Explicit and Implicit Intents

Project Name: IntentDemo

File 1: activity_main.xml

Location: app → res → layout → activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="20dp">

    <Button
        android:id="@+id/btnExplicit"
```

```
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Explicit Intent"/>

  <Button
    android:id="@+id/btnImplicit"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Implicit Intent"
    android:layout_marginTop="20dp"/>

</LinearLayout>
```

File 2: activity_second.xml

Create an Empty Views Activity named SecondActivity.

Location: app → res → layout → activity_second.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
  xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:gravity="center">

  <TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Welcome to Second Activity"
    android:textSize="24sp"/>
```

```
</LinearLayout>
```

File 3: MainActivity.java

Location: app → java → your package → MainActivity.java

```
package com.example.intentdemo;

import android.content.Intent;
import android.net.Uri;
import android.os.Bundle;
import android.widget.Button;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    Button btnExplicit, btnImplicit;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        btnExplicit = findViewById(R.id.btnExplicit);
        btnImplicit = findViewById(R.id.btnImplicit);

        // Explicit Intent
        btnExplicit.setOnClickListener(v -> {

            Intent intent = new Intent(
```

```
                MainActivity.this,
                SecondActivity.class
            );

            startActivity(intent);
        });

        // Implicit Intent
        btnImplicit.setOnClickListener(v -> {

            Intent intent = new Intent(
                Intent.ACTION_VIEW,
                Uri.parse("https://www.google.com")
            );

            startActivity(intent);
        });
    }
}
```

File 4: SecondActivity.java

```
package com.example.intentdemo;

import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;

public class SecondActivity extends AppCompatActivity {

    @Override
```

```
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_second);
    }
}
```

Note: Ensure SecondActivity is registered in AndroidManifest.xml. Android Studio normally adds it automatically when you create the activity.

PRACTICAL 7(b): Event Handling Using Listeners

Project Name: EventHandlingDemo

File 1: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="20dp">

    <Button
        android:id="@+id/btnClick"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Click Me"/>

    <Button
        android:id="@+id/btnLongClick"
```

```
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Long Press Me"
        android:layout_marginTop="20dp"/>

</LinearLayout>
```

File 2: MainActivity.java

```
package com.example.eventhandlingdemo;

import android.os.Bundle;
import android.widget.Button;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    Button btnClick, btnLongClick;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        btnClick = findViewById(R.id.btnClick);
        btnLongClick = findViewById(R.id.btnLongClick);

        // OnClickListener
```

```
        btnClick.setOnClickListener(v -> {

            Toast.makeText(
                    MainActivity.this,
                    "Button Clicked",
                    Toast.LENGTH_SHORT
            ).show();

        });

        // OnLongClickListener
        btnLongClick.setOnLongClickListener(v -> {

            Toast.makeText(
                    MainActivity.this,
                    "Button Long Pressed",
                    Toast.LENGTH_SHORT
            ).show();

            return true;
        });
    }
}
```

Output:

- Clicking **Click Me** displays Button Clicked.
- Long-pressing **Long Press Me** displays Button Long Pressed.

PRACTICAL 7(c): Adapters to Populate Spinner and ListView Dynamically

Project Name: AdapterDemo

File 1: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:orientation="vertical"
   android:padding="20dp">

   <Spinner
      android:id="@+id/spinner"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"/>

   <ListView
      android:id="@+id/listView"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:layout_marginTop="20dp"/>

</LinearLayout>
```

File 2: MainActivity.java

```
package com.example.adapterdemo;

import android.os.Bundle;
import android.view.View;
import android.widget.AdapterView;
```

```
import android.widget.ArrayAdapter;
import android.widget.ListView;
import android.widget.Spinner;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    Spinner spinner;
    ListView listView;

    String[] cities = {
            "Mumbai",
            "Pune",
            "Delhi",
            "Chennai",
            "Hyderabad"
    };

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        spinner = findViewById(R.id.spinner);
        listView = findViewById(R.id.listView);
```

```
// Spinner Adapter
ArrayAdapter<String> spinnerAdapter = new ArrayAdapter<>(
        this,
        android.R.layout.simple_spinner_dropdown_item,
        cities
);

spinner.setAdapter(spinnerAdapter);

// ListView Adapter
ArrayAdapter<String> listAdapter = new ArrayAdapter<>(
        this,
        android.R.layout.simple_list_item_1,
        cities
);

listView.setAdapter(listAdapter);

// Spinner Item Selection
spinner.setOnItemSelectedListener(
        new AdapterView.OnItemSelectedListener() {

            @Override
            public void onItemSelected(
                    AdapterView<?> parent,
                    View view,
                    int position,
                    long id) {
```

```
                Toast.makeText(
                        MainActivity.this,
                        "Selected: " + cities[position],
                        Toast.LENGTH_SHORT
                ).show();
            }

            @Override
            public void onNothingSelected(
                    AdapterView<?> parent) {
            }
        });

    // ListView Item Click
    listView.setOnItemClickListener(
            (parent, view, position, id) -> {

                Toast.makeText(
                        MainActivity.this,
                        "Clicked: " + cities[position],
                        Toast.LENGTH_SHORT
                ).show();
            });
  }
}
```

Output:

- The Spinner displays Mumbai, Pune, Delhi, Chennai and Hyderabad.

- The ListView displays the same five cities.
- Selecting a city in the Spinner displays a Selected: CityName message.
- Clicking a city in the ListView displays a Clicked: CityName message.

PRACTICAL 8(a): Started and Bound Services

Project Name: ServiceDemo

File 1: activity_main.xml

Location: app → res → layout → activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:orientation="vertical"
   android:gravity="center"
   android:padding="20dp">

   <Button
      android:id="@+id/btnStart"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Start Service"/>

   <Button
      android:id="@+id/btnStop"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Stop Service"
```

```
        android:layout_marginTop="15dp"/>

    <Button
        android:id="@+id/btnBound"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Bind Service"
        android:layout_marginTop="15dp"/>

</LinearLayout>
```

File 2: MainActivity.java

```
package com.example.servicedemo;

import android.content.ComponentName;
import android.content.Intent;
import android.content.ServiceConnection;
import android.os.Bundle;
import android.os.IBinder;
import android.widget.Button;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    Button btnStart, btnStop, btnBound;

    MyBoundService myBoundService;
```

```
boolean isBound = false;

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    btnStart = findViewById(R.id.btnStart);
    btnStop = findViewById(R.id.btnStop);
    btnBound = findViewById(R.id.btnBound);

    // Start Service
    btnStart.setOnClickListener(v -> {
        Intent intent = new Intent(
                MainActivity.this,
                MyStartedService.class
        );

        startService(intent);
    });

    // Stop Service
    btnStop.setOnClickListener(v -> {
        Intent intent = new Intent(
                MainActivity.this,
                MyStartedService.class
        );
```

```
        stopService(intent);
    });

    // Bind Service
    btnBound.setOnClickListener(v -> {
        if (!isBound) {
            Intent intent = new Intent(
                    MainActivity.this,
                    MyBoundService.class
            );

            bindService(
                    intent,
                    serviceConnection,
                    BIND_AUTO_CREATE
            );
        }
    });
}

// Service Connection
ServiceConnection serviceConnection = new ServiceConnection() {

    @Override
    public void onServiceConnected(
            ComponentName name,
            IBinder service) {
```

```
        MyBoundService.LocalBinder binder =
                (MyBoundService.LocalBinder) service;

        myBoundService = binder.getService();
        isBound = true;

        Toast.makeText(
                MainActivity.this,
                myBoundService.getMessage(),
                Toast.LENGTH_SHORT
        ).show();
    }

    @Override
    public void onServiceDisconnected(ComponentName name) {
        isBound = false;
    }
};

@Override
protected void onDestroy() {
    if (isBound) {
        unbindService(serviceConnection);
        isBound = false;
    }

    super.onDestroy();
}
```

```
}
```

File 3: MyStartedService.java

```
package com.example.servicedemo;

import android.app.Service;
import android.content.Intent;
import android.os.IBinder;
import android.widget.Toast;

public class MyStartedService extends Service {

   @Override
   public int onStartCommand(
         Intent intent,
         int flags,
         int startId) {

      Toast.makeText(
            this,
            "Started Service Running",
            Toast.LENGTH_SHORT
      ).show();

      return START_STICKY;
   }

   @Override
   public void onDestroy() {
```

```
        super.onDestroy();

        Toast.makeText(
                this,
                "Started Service Stopped",
                Toast.LENGTH_SHORT
        ).show();
    }

    @Override
    public IBinder onBind(Intent intent) {
        return null;
    }
}
```

File 4: MyBoundService.java

```
package com.example.servicedemo;

import android.app.Service;
import android.content.Intent;
import android.os.Binder;
import android.os.IBinder;

public class MyBoundService extends Service {

    private final IBinder binder = new LocalBinder();

    public class LocalBinder extends Binder {
```

```
        public MyBoundService getService() {
            return MyBoundService.this;
        }
    }

    @Override
    public IBinder onBind(Intent intent) {
        return binder;
    }

    public String getMessage() {
        return "Bound Service Connected";
    }
}
```

File 5: AndroidManifest.xml

Add both services inside the <application> element of your existing manifest. Keep your existing application settings and launcher activity.

```
<service
    android:name=".MyStartedService"
    android:exported="false" />

<service
    android:name=".MyBoundService"
    android:exported="false" />
```

Output:

- Clicking **Start Service** displays Started Service Running.
- Clicking **Stop Service** displays Started Service Stopped.
- Clicking **Bind Service** displays Bound Service Connected.

PRACTICAL 8(b): Notification Manager

Project Name: NotificationDemo

File 1: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:gravity="center">

   <Button
      android:id="@+id/btnNotify"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Show Notification"/>

</LinearLayout>
```

File 2: MainActivity.java

```
package com.example.notificationdemo;

import android.Manifest;
import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.content.pm.PackageManager;
import android.os.Build;
import android.os.Bundle;
import android.widget.Button;
```

```
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.NotificationCompat;

public class MainActivity extends AppCompatActivity {

    private static final String CHANNEL_ID = "MyChannel";
    private static final int NOTIFICATION_PERMISSION_CODE = 100;

    private Button btnNotify;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        btnNotify = findViewById(R.id.btnNotify);

        // Create notification channel
        createNotificationChannel();

        // Request permission for Android 13+
        requestNotificationPermission();

        // Button click
        btnNotify.setOnClickListener(v -> showNotification());
    }
```

```
// Request notification permission
private void requestNotificationPermission() {

    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {

        if (checkSelfPermission(
                Manifest.permission.POST_NOTIFICATIONS)
                != PackageManager.PERMISSION_GRANTED) {

            requestPermissions(
                    new String[]{
                            Manifest.permission.POST_NOTIFICATIONS
                    },
                    NOTIFICATION_PERMISSION_CODE
            );
        }
    }
}

// Create notification channel
private void createNotificationChannel() {

    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {

        NotificationChannel channel = new NotificationChannel(
                CHANNEL_ID,
                "My Channel",
                NotificationManager.IMPORTANCE_DEFAULT
```

```
        );

        channel.setDescription(
                "Notification Channel for Demo"
        );

        NotificationManager manager =
                getSystemService(NotificationManager.class);

        if (manager != null) {
            manager.createNotificationChannel(channel);
        }
    }
}

// Show notification
private void showNotification() {

    // Check permission for Android 13+
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {

        if (checkSelfPermission(
                Manifest.permission.POST_NOTIFICATIONS)
                != PackageManager.PERMISSION_GRANTED) {

            requestNotificationPermission();
            return;
        }
```

```
        }

        NotificationCompat.Builder builder =
                new NotificationCompat.Builder(this, CHANNEL_ID)
                        .setSmallIcon(
                                android.R.drawable.ic_dialog_info
                        )
                        .setContentTitle("Notification Example")
                        .setContentText(
                                "Hello from Android Notification"
                        )
                        .setPriority(
                                NotificationCompat.PRIORITY_DEFAULT
                        )
                        .setAutoCancel(true);

        NotificationManager manager =
                (NotificationManager)
                        getSystemService(NOTIFICATION_SERVICE);

        if (manager != null) {
            manager.notify(1, builder.build());
        }
    }
}
```

File 3: AndroidManifest.xml

Add the notification permission **above the <application> element** in your existing manifest.

<uses-permission

```
    android:name="android.permission.POST_NOTIFICATIONS" />
```

Output: A button named **Show Notification** appears. Clicking it displays a notification titled Notification Example with the message Hello from Android Notification, provided notifications are permitted.

Important: On Android 13 and later, allow notification permission when prompted. On older Android versions, the runtime permission request is not needed.

PRACTICAL 8(c): Static and Dynamic Broadcast Receivers

Project Name: BroadcastDemo

File 1: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Broadcast Receiver Example"
        android:textSize="22sp"/>

</LinearLayout>
```

File 2: MainActivity.java

```
package com.example.broadcastdemo;

import android.content.IntentFilter;
import android.os.Bundle;
```

```
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.content.ContextCompat;

public class MainActivity extends AppCompatActivity {

    MyReceiver myReceiver = new MyReceiver();

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        // Dynamic Receiver
        IntentFilter filter = new IntentFilter();
        filter.addAction(
                "android.intent.action.AIRPLANE_MODE"
        );

        ContextCompat.registerReceiver(
                this,
                myReceiver,
                filter,
                ContextCompat.RECEIVER_EXPORTED
        );
    }

    @Override
```

```
    protected void onDestroy() {
        unregisterReceiver(myReceiver);
        super.onDestroy();
    }
}
```

File 3: MyReceiver.java

```
package com.example.broadcastdemo;

import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.widget.Toast;

public class MyReceiver extends BroadcastReceiver {

    @Override
    public void onReceive(Context context, Intent intent) {

        Toast.makeText(
                context,
                "Broadcast Received",
                Toast.LENGTH_SHORT
        ).show();
    }
}
```

File 4: AndroidManifest.xml

Add the receiver declaration inside the <application> element of your existing manifest.

```
<receiver
```

```
android:name=".MyReceiver"
android:exported="true">

<intent-filter>
  <action
    android:name="android.intent.action.AIRPLANE_MODE" />
</intent-filter>

</receiver>
```

Output: The screen displays Broadcast Receiver Example. The receiver displays Broadcast Received when it receives the configured broadcast, subject to Android's broadcast restrictions.

Important distinction:

- **Static receiver:** Declared in AndroidManifest.xml.
- **Dynamic receiver:** Registered in Java code while the application is running.

The PDF demonstrates both declarations using the same MyReceiver class. For a cleaner test of dynamic registration, use only the Java registration and remove the manifest <receiver> declaration. This avoids registering the same receiver through both methods.

PRACTICAL 9(a): SQLite Database CRUD Operations

Project Name: SQLiteCRUDDemo

CRUD stands for Create, Read, Update and Delete.

File 1: activity_main.xml

Location: app → res → layout → activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
  xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
```

```
android:orientation="vertical"
android:padding="20dp">

<EditText
   android:id="@+id/etName"
   android:layout_width="match_parent"
   android:layout_height="wrap_content"
   android:hint="Enter Name"/>

<EditText
   android:id="@+id/etNewName"
   android:layout_width="match_parent"
   android:layout_height="wrap_content"
   android:hint="Enter New Name"/>

<Button
   android:id="@+id/btnInsert"
   android:layout_width="match_parent"
   android:layout_height="wrap_content"
   android:text="Insert"/>

<Button
   android:id="@+id/btnView"
   android:layout_width="match_parent"
   android:layout_height="wrap_content"
   android:text="View Data"/>

<Button
```

```
        android:id="@+id/btnUpdate"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Update"/>

    <Button
        android:id="@+id/btnDelete"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Delete"/>

    <TextView
        android:id="@+id/txtResult"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="18sp"
        android:paddingTop="20dp"/>

</LinearLayout>
```

File 2: MainActivity.java

```
package com.example.sqlitecruddemo;

import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import android.widget.Toast;
```

```
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    EditText etName, etNewName;
    TextView txtResult;

    Button btnInsert, btnView, btnUpdate, btnDelete;

    DBHelper dbHelper;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        etName = findViewById(R.id.etName);
        etNewName = findViewById(R.id.etNewName);
        txtResult = findViewById(R.id.txtResult);

        btnInsert = findViewById(R.id.btnInsert);
        btnView = findViewById(R.id.btnView);
        btnUpdate = findViewById(R.id.btnUpdate);
        btnDelete = findViewById(R.id.btnDelete);

        dbHelper = new DBHelper(this);

        // Insert
```

```
btnInsert.setOnClickListener(v -> {

    String name = etName.getText().toString().trim();

    if (name.isEmpty()) {
        etName.setError("Enter Name");
        return;
    }

    dbHelper.insertData(name);

    Toast.makeText(
            this,
            "Data Inserted",
            Toast.LENGTH_SHORT
    ).show();

    etName.setText("");
});

// View
btnView.setOnClickListener(v -> {

    String data = dbHelper.viewData();
    txtResult.setText(data);
});

// Update
```

```
btnUpdate.setOnClickListener(v -> {

    String oldName = etName.getText().toString().trim();
    String newName = etNewName.getText().toString().trim();

    if (oldName.isEmpty() || newName.isEmpty()) {
        etName.setError("Enter Old Name");
        etNewName.setError("Enter New Name");
        return;
    }

    dbHelper.updateData(oldName, newName);

    Toast.makeText(
            this,
            "Data Updated",
            Toast.LENGTH_SHORT
    ).show();
});

// Delete
btnDelete.setOnClickListener(v -> {

    String name = etName.getText().toString().trim();

    if (name.isEmpty()) {
        etName.setError("Enter Name");
        return;
```

```
            }

            dbHelper.deleteData(name);

            Toast.makeText(
                    this,
                    "Data Deleted",
                    Toast.LENGTH_SHORT
            ).show();
        });
    }
}
```

File 3: DBHelper.java

```
package com.example.sqlitecruddemo;

import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;

public class DBHelper extends SQLiteOpenHelper {

    public DBHelper(Context context) {
        super(context, "StudentDB", null, 1);
    }

    @Override
```

```
public void onCreate(SQLiteDatabase db) {
   db.execSQL("CREATE TABLE student(name TEXT)");
}

@Override
public void onUpgrade(
       SQLiteDatabase db,
       int oldVersion,
       int newVersion) {

   db.execSQL("DROP TABLE IF EXISTS student");
   onCreate(db);
}

// Insert
public void insertData(String name) {

   SQLiteDatabase db = this.getWritableDatabase();

   ContentValues values = new ContentValues();
   values.put("name", name);

   db.insert("student", null, values);
}

// View
public String viewData() {
```

```
    SQLiteDatabase db = this.getReadableDatabase();

    Cursor cursor = db.rawQuery(
            "SELECT * FROM student",
            null
    );

    StringBuilder data = new StringBuilder();

    while (cursor.moveToNext()) {
        data.append(cursor.getString(0)).append("\n");
    }

    cursor.close();

    return data.toString();
}

// Update
public void updateData(String oldName, String newName) {

    SQLiteDatabase db = this.getWritableDatabase();

    ContentValues values = new ContentValues();
    values.put("name", newName);

    db.update(
            "student",
```

```
            values,
            "name=?",
            new String[]{oldName}
        );
    }

    // Delete
    public void deleteData(String name) {

        SQLiteDatabase db = this.getWritableDatabase();

        db.delete(
            "student",
            "name=?",
            new String[]{name}
        );
    }
}
```

Output:

- Insert: Adds a name to the database.
- View Data: Displays saved names.
- Update: Changes an existing name.
- Delete: Removes a name from the database.

PRACTICAL 9(b): SQLite Database with ListView

Project Name: SQLiteListViewDemo

This application inserts student names into an SQLite database and displays them in a ListView.

File 1: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:orientation="vertical"
   android:padding="20dp">

   <EditText
      android:id="@+id/etName"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:hint="Enter Name"/>

   <Button
      android:id="@+id/btnInsert"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:text="Insert"/>

   <Button
      android:id="@+id/btnView"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:text="View Data"/>

   <ListView
```

```
    android:id="@+id/listView"
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1"/>

</LinearLayout>
```

File 2: MainActivity.java

```
package com.example.sqlitelistviewdemo;

import android.os.Bundle;
import android.widget.ArrayAdapter;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ListView;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

import java.util.ArrayList;

public class MainActivity extends AppCompatActivity {

   EditText etName;
   Button btnInsert, btnView;
   ListView listView;

   DBHelper dbHelper;
```

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    etName = findViewById(R.id.etName);
    btnInsert = findViewById(R.id.btnInsert);
    btnView = findViewById(R.id.btnView);
    listView = findViewById(R.id.listView);

    dbHelper = new DBHelper(this);

    // Insert Data
    btnInsert.setOnClickListener(v -> {

        String name = etName.getText().toString().trim();

        if (name.isEmpty()) {
            etName.setError("Enter Name");
            return;
        }

        dbHelper.insertData(name);

        Toast.makeText(
                this,
                "Data Inserted",
                Toast.LENGTH_SHORT
```

```
            ).show();

            etName.setText("");
        });

        // View Data
        btnView.setOnClickListener(v -> {

            ArrayList<String> list = dbHelper.getData();

            ArrayAdapter<String> adapter = new ArrayAdapter<>(
                    this,
                    android.R.layout.simple_list_item_1,
                    list
            );

            listView.setAdapter(adapter);
        });
    }
}
```

File 3: DBHelper.java

```
package com.example.sqlitelistviewdemo;

import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;
```

```
import java.util.ArrayList;

public class DBHelper extends SQLiteOpenHelper {

    public DBHelper(Context context) {
        super(context, "StudentDB", null, 1);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL("CREATE TABLE student(name TEXT)");
    }

    @Override
    public void onUpgrade(
            SQLiteDatabase db,
            int oldVersion,
            int newVersion) {
        // Database upgrade code
    }

    // Insert Data
    public void insertData(String name) {

        SQLiteDatabase db = this.getWritableDatabase();

        ContentValues values = new ContentValues();
```

```
        values.put("name", name);

        db.insert("student", null, values);
    }

    // Get All Data
    public ArrayList<String> getData() {

        ArrayList<String> list = new ArrayList<>();

        SQLiteDatabase db = this.getReadableDatabase();

        Cursor cursor = db.rawQuery(
                "SELECT name FROM student",
                null
        );

        while (cursor.moveToNext()) {
            list.add(cursor.getString(0));
        }

        cursor.close();

        return list;
    }
}
```

Output:

1. Enter a student name.

2. Click **Insert** to save it.
3. Click **View Data** to display the saved names in the ListView.

PRACTICAL 9(c): Fetch JSON Data from a Web API

Project Name: JSONDemo

This application fetches JSON data from a web API and displays the title and body.

File 1: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:orientation="vertical"
   android:padding="20dp">

   <Button
      android:id="@+id/btnFetch"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:text="Fetch JSON Data"/>

   <TextView
      android:id="@+id/txtResult"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:layout_marginTop="20dp"
      android:textSize="18sp"
      android:textColor="#000000"/>
```

```
</LinearLayout>
```

File 2: MainActivity.java

```
package com.example.jsondemo;

import android.os.Bundle;
import android.widget.Button;
import android.widget.TextView;

import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.net.HttpURLConnection;
import java.net.URL;

import org.json.JSONObject;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    Button btnFetch;
    TextView txtResult;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
```

```
    btnFetch = findViewById(R.id.btnFetch);
    txtResult = findViewById(R.id.txtResult);

    btnFetch.setOnClickListener(v -> fetchData());
}

private void fetchData() {

    new Thread(() -> {

        HttpURLConnection connection = null;

        try {

            // Web Service URL
            URL url = new URL(
                    "https://jsonplaceholder.typicode.com/posts/1"
            );

            connection = (HttpURLConnection) url.openConnection();

            connection.setRequestMethod("GET");
            connection.setConnectTimeout(10000);
            connection.setReadTimeout(10000);

            BufferedReader reader = new BufferedReader(
                    new InputStreamReader(
                            connection.getInputStream()
```

```
        )
);

StringBuilder result = new StringBuilder();
String line;

while ((line = reader.readLine()) != null) {
    result.append(line);
}

reader.close();

// JSON Parsing
JSONObject jsonObject =
        new JSONObject(result.toString());

String title = jsonObject.getString("title");
String body = jsonObject.getString("body");

// Display Data
runOnUiThread(() -> {

    txtResult.setText(
            "Title: " + title +
            "\n\nBody: " + body
    );
});
```

```
        } catch (Exception e) {

            runOnUiThread(() -> {
                txtResult.setText(
                        "Error: " + e.getMessage()
                );
            });

        } finally {

            if (connection != null) {
                connection.disconnect();
            }
        }

    }).start();
  }
}
```

File 3: AndroidManifest.xml

Add the Internet permission **above the <application> element** in your existing manifest.

```
<uses-permission android:name="android.permission.INTERNET"/>
```

Output:

- The screen displays a **Fetch JSON Data** button.
- Clicking it requests data from the API.
- The application's TextView displays the JSON title and body.

Important: The device or emulator needs an internet connection for this practical to work.

ANDROID PRACTICAL 10 — ALL CODES

Practical 10(a): Threads and Handlers for Background Processing

Project Name: ThreadHandlerDemo

1. activity_main.xml

Path: app > res > layout > activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:orientation="vertical"
   android:gravity="center"
   android:padding="20dp">

   <Button
      android:id="@+id/btnStart"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Start Background Task" />

   <TextView
      android:id="@+id/txtResult"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:textSize="22sp"
      android:paddingTop="20dp" />

</LinearLayout>
```

2. MainActivity.java

Path: app > java > com.example.threadhandlerdemo > MainActivity.java

```
package com.example.threadhandlerdemo;

import android.os.Bundle;
import android.os.Handler;
import android.os.Looper;
import android.widget.Button;
import android.widget.TextView;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    Button btnStart;
    TextView txtResult;

    Handler handler = new Handler(Looper.getMainLooper());

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        btnStart = findViewById(R.id.btnStart);
        txtResult = findViewById(R.id.txtResult);

        btnStart.setOnClickListener(v -> {
```

```
        new Thread(() -> {

            try {
                Thread.sleep(3000);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }

            handler.post(() -> {
                txtResult.setText("Background Task Completed");
            });

        }).start();

    });
  }
}
```

Output: A button appears. When clicked, the app waits approximately 3 seconds and displays **Background Task Completed**.

Theory: A Thread performs tasks in the background, while a Handler sends the result back to the main UI thread.

Practical 10(b): Media APIs and Telephone APIs

Project Name: MediaTelePhoneDemo

1. activity_main.xml

Path: app > res > layout > activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
```

```
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="20dp">

    <Button
        android:id="@+id/btnPlay"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Play Music" />

    <Button
        android:id="@+id/btnDial"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Open Dialer"
        android:layout_marginTop="20dp" />

</LinearLayout>
```

2. MainActivity.java

Path: app > java > com.example.mediatelephonedemo > MainActivity.java

```
package com.example.mediatelephonedemo;

import android.content.Intent;
import android.media.MediaPlayer;
import android.net.Uri;
```

```
import android.os.Bundle;
import android.widget.Button;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    Button btnPlay, btnDial;
    MediaPlayer mediaPlayer;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        btnPlay = findViewById(R.id.btnPlay);
        btnDial = findViewById(R.id.btnDial);

        mediaPlayer = MediaPlayer.create(this, R.raw.music);

        btnPlay.setOnClickListener(v -> {
            if (mediaPlayer != null) {
                mediaPlayer.start();
            }
        });

        btnDial.setOnClickListener(v -> {
            Intent intent = new Intent(Intent.ACTION_DIAL);
```

```
        intent.setData(Uri.parse("tel:9876543210"));
        startActivity(intent);
    });
}

@Override
protected void onDestroy() {
    super.onDestroy();

    if (mediaPlayer != null) {
        mediaPlayer.release();
        mediaPlayer = null;
    }
}
}
```

3. Required Music File

To play music, you must add an audio file to the project.

1. Open app > src > main > res.
2. Create a folder named raw if it does not already exist.
3. Place your audio file inside it.
4. Rename the file to music.mp3.

Your file path should be:

app/src/main/res/raw/music.mp3

Output:

- **Play Music:** Plays the audio file.
- **Open Dialer:** Opens the phone dialer with the example number filled in.

Theory: Media APIs handle audio and video playback. Telephone APIs allow applications to open telephone-related functions using Android intents.

Important: ACTION_DIAL opens the dialer; it does not make a call automatically.

Practical 10(c): Runtime Permission and Security Handling in AndroidManifest.xml

Project Name: PermissionDemo

1. activity_main.xml

Path: app > res > layout > activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:orientation="vertical"
   android:gravity="center"
   android:padding="20dp">

   <Button
      android:id="@+id/btnPermission"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Request Camera Permission" />

</LinearLayout>
```

2. MainActivity.java

Path: app > java > com.example.permissiondemo > MainActivity.java

```
package com.example.permissiondemo;

import android.content.pm.PackageManager;
import android.os.Bundle;
```

```
import android.widget.Button;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;

public class MainActivity extends AppCompatActivity {

    Button btnPermission;
    private static final int CAMERA_PERMISSION_CODE = 1;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        btnPermission = findViewById(R.id.btnPermission);

        btnPermission.setOnClickListener(v -> checkPermission());
    }

    private void checkPermission() {

        if (ContextCompat.checkSelfPermission(
                this,
                android.Manifest.permission.CAMERA
        ) == PackageManager.PERMISSION_GRANTED) {
```

```
        Toast.makeText(
                this,
                "Permission Already Granted",
                Toast.LENGTH_SHORT
        ).show();

    } else {

        ActivityCompat.requestPermissions(
                this,
                new String[]{
                        android.Manifest.permission.CAMERA
                },
                CAMERA_PERMISSION_CODE
        );
    }
}

@Override
public void onRequestPermissionsResult(
        int requestCode,
        String[] permissions,
        int[] grantResults
) {
    super.onRequestPermissionsResult(
            requestCode,
            permissions,
```

```
            grantResults
        );

        if (requestCode == CAMERA_PERMISSION_CODE) {

            if (grantResults.length > 0 &&
                grantResults[0] == PackageManager.PERMISSION_GRANTED) {

                Toast.makeText(
                        this,
                        "Permission Granted",
                        Toast.LENGTH_SHORT
                ).show();

            } else {

                Toast.makeText(
                        this,
                        "Permission Denied",
                        Toast.LENGTH_SHORT
                ).show();
            }
        }
    }
}
```

3. AndroidManifest.xml

Path: app > manifests > AndroidManifest.xml

Add the camera permission **before the <application> tag**, inside the <manifest> element:

<uses-permission android:name="android.permission.CAMERA" />

For example:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-permission android:name="android.permission.CAMERA" />

    <application
        android:allowBackup="true"
        android:label="PermissionDemo"
        android:supportsRtl="true"
        android:theme="@style/Theme.PermissionDemo">

        <activity
            android:name=".MainActivity"
            android:exported="true">

            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>

        </activity>

    </application>

</manifest>
```

Note: If your existing manifest has different theme or application settings, keep those settings and add only the permission line. Do not replace the entire manifest unnecessarily.

Output: When the button is clicked, Android asks the user for camera permission. A toast displays whether permission was granted or denied.

Theory: Runtime permissions protect sensitive features such as the camera. The permission is declared in AndroidManifest.xml and requested from the user at runtime.
